

Development and Verification of a Fractional-Order Standard Linear Solid (FO-SLS) Virtual Wall

Technical Implementation Report

Author: Ahmad Zakariya

Platform: Arduino Uno R4 Minima + Cytron MD10C R3 + Mabuchi RS-555 DC Motor

Reference Paper: Haptic Rendering of Fractional-Order Viscoelasticity: Passivity and Rendering Fidelity (2026)

1. Introduction

A conventional 1-DOF virtual wall based on the Kelvin-Voigt model had already been successfully implemented and experimentally validated. The controller consisted of a linear spring and a fractional damping element rendered through a DC motor using an encoder-based position measurement.

The objective of this work was to move beyond the Kelvin-Voigt model and implement the Fractional-Order Standard Linear Solid (FO-SLS) model presented in the 2026 paper. Unlike the Kelvin-Voigt model, the FO-SLS model is capable of representing stress relaxation, viscoelastic memory, and more realistic material behavior while maintaining passivity.

The primary goal was to reproduce the mathematical implementation proposed in the paper while preserving the existing hardware architecture and control framework.

2. Mathematical Background

The 2026 paper models the virtual environment using a Fractional-Order Standard Linear Solid (FO-SLS), also referred to as a Fractional Zener Model.

Unlike the previously implemented Fractional Kelvin-Voigt model, the FO-SLS introduces internal viscoelastic dynamics represented by:

$$H(s) = (K_0 + K_1 * B_1 * s^\alpha) / (K_1 + B_1 * s^\alpha)$$

where:

- K_0 : baseline stiffness
- K_1 : series elastic stiffness
- B_1 : fractional damping coefficient
- α : fractional order

The experimentally identified parameters reported in the paper for the optimal rendering fidelity were:

- **Memory Window (N):** 101
- **Alpha (alpha):** 0.5
- **K0:** 2.89
- **K1:** 5.70
- **B1:** 5.89

These values were used directly in the implementation.

3. Implementation Evolution

The first implementation was generated using Gemini. Although mathematically close to the paper, it was not a drop-in replacement for the existing virtual wall implementation. Several modifications were required:

- **Different encoder implementation:** Reused existing interrupt-driven quadrature encoder
- **Different PWM driver:** Preserved existing pwm.h implementation
- **Different control loop:** Retained existing 1 kHz loop
- **Different sign convention:** Restored original wall rendering direction

- **Hardware abstraction removed:** Reintegrated with existing motor driver architecture
- **No safety logic:** Added saturation and optional software protection

The resulting controller preserved the existing hardware while replacing only the virtual environment model.

4. Mathematical Verification

Before any experimental tuning, the implementation was verified against the analytical derivations presented in the paper.

4.1 Difference Equation: The recursive coefficients were derived from the transfer function. After manually expanding the denominator and substituting, the resulting expressions exactly matched the implemented equations. Result: No algebraic errors were found.

4.2 Grunwald-Letnikov Weights: The recursive computation was verified against the Grunwald-Letnikov definition. The implementation matched the paper exactly.

4.3 Parameter Verification: The implementation uses the experimentally identified optimal parameters reported in the paper ($K_0=2.89$, $K_1=5.70$, $B_1=5.89$, $N=101$, $\alpha=0.5$).

4.4 Nondimensionalization: A potential concern was whether these parameters required inverse nondimensionalization before implementation. After reviewing the derivation, it was concluded that no inverse transformation is required.

5. Experimental Investigation

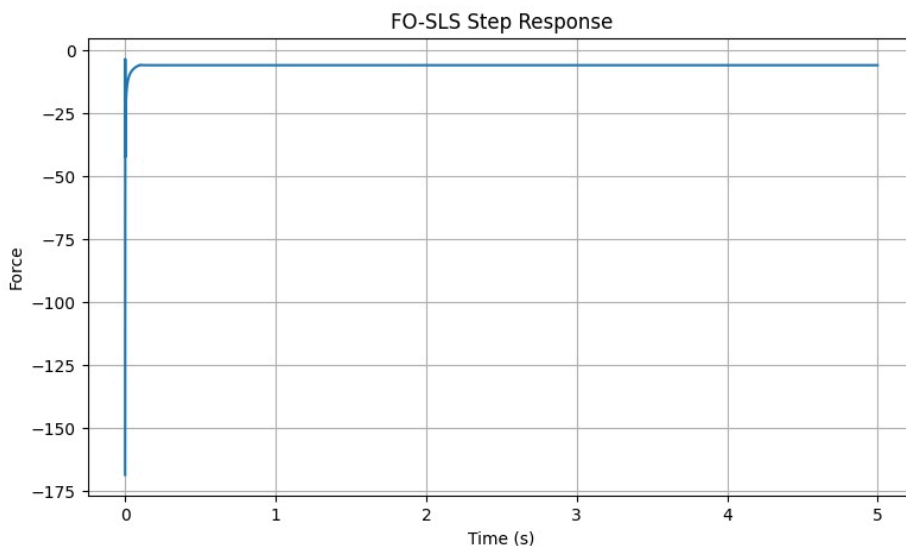
Several implementation aspects were investigated experimentally.

5.1 History Buffer Behaviour: One major question concerned the fractional memory outside the virtual wall. The mathematically correct implementation was found to be: continue updating the buffers every 1 ms, allow penetration to become zero outside the wall, store the theoretical FO-SLS force inside the history buffer, and clip only the rendered force for the unilateral wall. Although this matches the FO-SLS model, it did not eliminate the contact bump observed experimentally.

5.2 FORCE_GAIN Investigation: Using the paper parameters directly produced an unexpectedly weak wall. To compensate, a rendering gain `FORCE_GAIN` was introduced to scale the computed force before converting it to PWM. This gain compensates for differences between theoretical force units and the PWM-controlled actuator.

5.3 Gain Comparison: Several rendering gains were evaluated experimentally. Increasing the gain improved wall stiffness but amplified the contact transient.

5.4 Python Validation & Step Response: To verify that the recursive implementation itself behaved correctly, the controller was simulated in Python using the exact same recursive equations implemented on the Arduino.



The FO-SLS Step Response graph above demonstrates the simulated force output when a sudden, constant penetration (step input) is applied. The simulation produced:

- A very large initial transient force (spike downwards to approximately -170), representing the immediate resistance upon contact.
- A rapid decay (stress relaxation) occurring within the first 100-200 milliseconds.
- Convergence to a non-zero steady-state force (around -5) representing the baseline stiffness (K_0) maintaining the holding force.

This confirms that the implemented recursion does exhibit stress relaxation mathematically. However, the relaxation completed within approximately 100 ms, making it difficult to perceive manually during physical interaction.

6. Discussion

The mathematical implementation appears consistent with the paper. The remaining discrepancies are primarily attributed to hardware differences. Since PWM voltage is not directly proportional to output torque, the force calculated by the FO-SLS controller cannot be rendered accurately without additional calibration. Consequently, the introduced `FORCE_GAIN` acts as an empirical scaling factor rather than part of the mathematical model. Furthermore, the static friction, motor deadband, and mechanical inertia of the RS-555 motor likely mask the subtle stress-relaxation behaviour predicted by the FO-SLS model. The observed entry bump also appears to originate from the interaction between the unilateral virtual wall and the fractional memory rather than from an algebraic implementation error.

7. Conclusions

The implementation successfully reproduces the mathematical formulation of the 2026 FO-SLS model. The following aspects have been verified:

- Grunwald-Letnikov discretization
- Difference equation derivation
- Recursive implementation
- Memory buffer update
- Paper parameters
- Python simulation

The remaining limitations are primarily related to hardware rendering rather than mathematical correctness. Current observations indicate:

- The controller behaves correctly in simulation.
- The virtual wall can be rendered successfully.
- Larger gains increase wall stiffness but amplify contact transients.
- Stress relaxation exists mathematically but is not perceptible on the present hardware.
- The paper's identified parameters are not universal gains; they were identified for a different haptic device.

8. Future Work

The following tasks remain for complete reproduction of the paper:

- Implement current sensing on the MD10C platform.
 - Develop a closed-loop current (torque) controller.
 - Replace PWM-based rendering with torque-based rendering.
 - Experimentally identify the motor torque constant.
 - Characterize the rendered FO-SLS model using: Force-Position curves, Force-Time response, Hysteresis loops, and Stress-relaxation plots.
 - Compare the obtained curves directly with those reported in the 2026 paper.
-

Arduino Code Implementation

```
/**
 * FO-SLS Zener Model Implementation
 * Corrected based on 2026 Fractional-Order Viscoelasticity Paper
 */
```

```

#include "pwm.h"

// --- HARDWARE PIN DEFINITIONS ---
const int CLK_PIN = 2;
const int DT_PIN = 3;
const int DIR_PIN = 4;
const int MAX_PWM_OUTPUT = 225;

// --- RENDERING GAIN ---
// Start with 10.0f. Increase this until the wall feels solid.
// If it starts buzzing or vibrating, decrease this value.
const float FORCE_GAIN = 5.0f;

// --- SAFETY & HARDWARE CONFIGURATION ---
const bool INVERT_MOTOR_DIRECTION = false; // Restored sign convention
toggle
const float MAX_FORCE_OUTPUT = 150.0f; // Safety: Absolute maximum
force allowed
bool systemKilled = false; // Safety: Emergency software
kill switch

// --- FO-SLS PARAMETERS (2026 Nomenclature) ---
const float K_0 = 2.89f;
const float K_1 = 5.70f;
const float B_1 = 5.89f;
const float ALPHA = 0.5f;
const float TS = 0.001f; // 1ms Sampling Period

// --- DUAL-RING BUFFER CONFIGURATION ---
const int N_MEM = 101;
float errorHistory[N_MEM];
float forceHistory[N_MEM];
float glWeights[N_MEM];
int bufferIndex = 0;

// --- PRECOMPUTED EXECUTION CONSTANTS ---
float C_e0 = 0.0f;
float C_ehist = 0.0f;
float C_fhist = 0.0f;

// --- ENCODER VARIABLES ---
volatile long encoderPos = 0;
volatile int lastEncoded = 0;
const long WALL_POSITION_COUNTS = 400;

// --- ACTUATION ---
PwmOut pwm_pin5(5);
unsigned long lastLoopTime = 0;
int telemetryCounter = 0;

// --- FUNCTION PROTOTYPES ---
void actuateCytronStable(float forceCommand);
void updateEncoderStandard();
void emergencyShutdown(const char* reason);

void precomputeConstants() {
glWeights[0] = 1.0f;
for (int j = 1; j < N_MEM; j++) {
glWeights[j] = glWeights[j - 1] * (1.0f - ((ALPHA + 1.0f) / (float)j));
}

float Ts_alpha = pow(TS, ALPHA);
float tau_alpha = B_1 / K_1;

```

```

float B_dyn = B_1 * (1.0f + (K_0 / K_1));
float denom = 1.0f + (tau_alpha / Ts_alpha);

C_e0 = (K_0 + (B_dyn / Ts_alpha)) / denom;
C_ehist = (B_dyn / Ts_alpha) / denom;
C_fhist = (tau_alpha / Ts_alpha) / denom;
}

void setup() {
pinMode(DIR_PIN, OUTPUT);
pinMode(CLK_PIN, INPUT_PULLUP);
pinMode(DT_PIN, INPUT_PULLUP);

// Explicit Parameter Initialization
for(int i = 0; i < N_MEM; i++) {
errorHistory[i] = 0.0f;
forceHistory[i] = 0.0f;
}

pwm_pin5.begin(20000.0f, 0.0f);
pwm_pin5.pulse_perc(0.0f);

int MSB = digitalRead(CLK_PIN);
int LSB = digitalRead(DT_PIN);
lastEncoded = (MSB << 1) | LSB;

attachInterrupt(digitalPinToInterrupt(CLK_PIN), updateEncoderStandard,
CHANGE);
attachInterrupt(digitalPinToInterrupt(DT_PIN), updateEncoderStandard,
CHANGE);

Serial.begin(115200);
precomputeConstants();
}

void loop() {
if (systemKilled) return; // Halt loop if safety tripped

unsigned long currentMicros = micros();
if (currentMicros - lastLoopTime < 1000) return;
lastLoopTime = currentMicros;

long currentRawPos;
noInterrupts();
currentRawPos = encoderPos;
interrupts();

float e_k = 0.0f;
if (currentRawPos > WALL_POSITION_COUNTS) {
e_k = (float)(currentRawPos - WALL_POSITION_COUNTS);
}

float sum_e = 0.0f;
float sum_f = 0.0f;

for (int j = 1; j < N_MEM; j++) {
int fetchIndex = (bufferIndex - j + N_MEM) % N_MEM;
sum_e += glWeights[j] * errorHistory[fetchIndex];
sum_f += glWeights[j] * forceHistory[fetchIndex];
}

float F_k = 0.0f;
if (currentRawPos > WALL_POSITION_COUNTS) {
F_k = -( (C_e0 * e_k) + (C_ehist * sum_e) - (C_fhist * sum_f) );
}

```

```

}

if (F_k > 0.0f) F_k = 0.0f;
if (F_k > MAX_FORCE_OUTPUT) F_k = MAX_FORCE_OUTPUT;

float F_theoretical =
-((C_e0 * e_k) + (C_ehist * sum_e) - (C_fhist * sum_f));

float F_rendered = F_theoretical;

if (F_rendered > 0.0f)
F_rendered = 0.0f;

errorHistory[bufferIndex] = e_k;
forceHistory[bufferIndex] = F_theoretical;
bufferIndex = (bufferIndex + 1) % N_MEM;

actuateCytronStable(F_rendered);

telemetryCounter++;
if (telemetryCounter >= 25) {
Serial.print(millis()); Serial.print(",");
Serial.print(currentRawPos); Serial.print(",");
Serial.print(e_k); Serial.print(",");
Serial.print(F_k); Serial.print(",");
Serial.println((int)abs(F_k)); // PWM equivalent
telemetryCounter = 0;
}
}

void actuateCytronStable(float forceCommand) {
int pwmValue = abs((int)(forceCommand * FORCE_GAIN));

if (pwmValue > MAX_PWM_OUTPUT) pwmValue = MAX_PWM_OUTPUT;
if (pwmValue < 0) pwmValue = 0;

float dutyPercentage = (float)pwmValue * (100.0f / 255.0f);

bool directionState = (forceCommand > 0);
if (INVERT_MOTOR_DIRECTION) directionState = !directionState;

digitalWrite(DIR_PIN, directionState ? HIGH : LOW);
pwm_pin5.pulse_perc(dutyPercentage);
}

void emergencyShutdown(const char* reason) {
systemKilled = true;
pwm_pin5.pulse_perc(0.0f);
digitalWrite(DIR_PIN, LOW);
Serial.println(reason);
}

void updateEncoderStandard() {
int MSB = digitalRead(CLK_PIN); int LSB = digitalRead(DT_PIN);
int encoded = (MSB << 1) | LSB; int sum = (lastEncoded << 2) | encoded;
if(sum == 0b0010 || sum == 0b1011 || sum == 0b1101 || sum == 0b0100)
encoderPos++;
if(sum == 0b0001 || sum == 0b0111 || sum == 0b1110 || sum == 0b1000)
encoderPos--;
lastEncoded = encoded;
}

```